

NDNSF-DI

ACK-driven ONNX placement and NDN pull-based distributed inference

MODEL-FAMILY NEUTRAL

ACK-DRIVEN PLANNING

DATA-DRIVEN EXECUTION

The Problem: A Request Does Not Contain a Deployment

The application knows

- ▶ immutable model identity;
- ▶ task, input, options, objective;
- ▶ deadline and policy constraints.

Only the runtime can discover

- ▶ available Providers and GPU envelopes;
- ▶ queue, RTT, bandwidth, cache residency;
- ▶ a graph-valid split and role assignment.

Key point

Provider discovery must precede the final split and placement decision.

Next: define the design goals and non-goals

Design Goals and Explicit Non-Goals

Goals

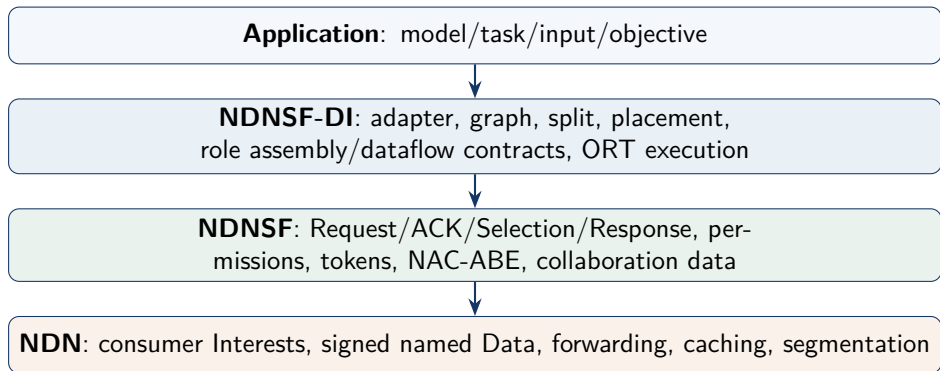
- ▶ one durable application invocation;
- ▶ replaceable placement policy;
- ▶ model-family-neutral adapters;
- ▶ graph- and capacity-safe splitting;
- ▶ authenticated, replay-safe execution;
- ▶ complete failure and recovery states.

Non-goals

- ▶ no model semantics in base NDNF;
- ▶ no global “all Providers ready” barrier;
- ▶ no trust in unsigned resource claims;
- ▶ no approximate cache-key matching;
- ▶ no performance claim from a control-plane smoke.

Next: place each responsibility in the correct layer

Ownership Boundary: NDNF Carries, NDNF-DI Interprets



Key point

Cross-Provider activations and tensor collectives remain NDN Interest/Data; ONNX Runtime performs only Provider-local compute.

Core Vocabulary and Immutable Identities

Invocation	One application-visible operation with one handle and terminal result.
Attempt	One bounded network decision generation inside an invocation.
ACK-closed snapshot	Immutable authenticated candidate set and its canonical digest.
Plan commit	One-shot generic projection of roles, dependencies, Providers, and opaque assignments.
Selection	ProviderToken-authorized final assignment sent to each chosen Provider.
Generation	Role execution epoch that fences late data and superseded work.

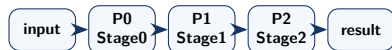
Key point

Identity and state transitions, not timing assumptions, determine which work is current.

Next: start from the public application call

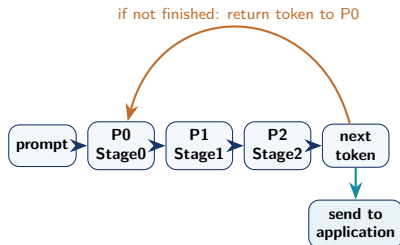
Single-Pass and Autoregressive Models Use Different Flows

Single-pass model (for example, YOLO)



- ▶ One invocation moves from P0 to P2 once. P2 returns the final result.
- ▶ No output feeds back into another model pass.

Autoregressive LLM generation

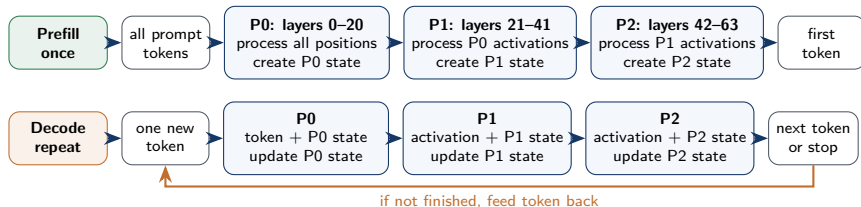


- ▶ Prefill sends the full prompt from P0 to P2 once.
- ▶ P2 publishes each new token to the application and feeds it back to P0. The token then passes through P0, P1, and P2 again.

Key point

A single-pass invocation crosses the pipeline once. Autoregressive generation repeats the pipeline for each new token.

Prefill and Decode in Our Three-Provider Example



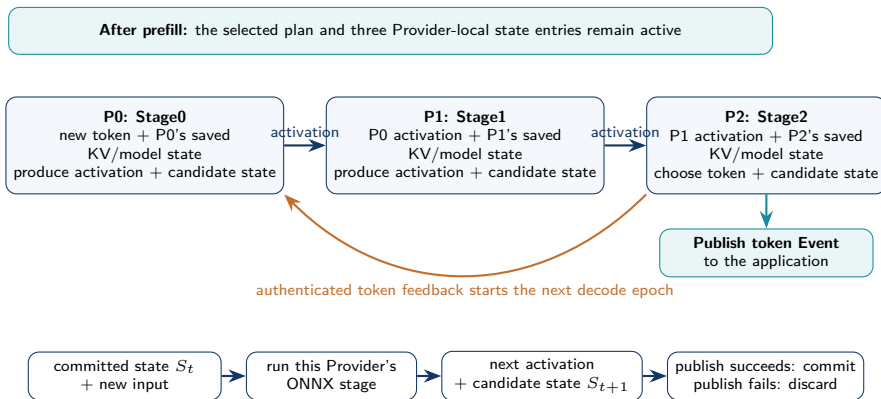
- ▶ **Prefill** handles all prompt positions once, creates each Provider's layer-specific state, and produces the first token.
- ▶ **Decode** handles one new token per round, reuses and updates those three states, and produces the next token.
- ▶ The Providers and layer ranges stay fixed; NDN carries tokens and activations, while saved state remains local.

Key point

Prefill runs once; decode repeats through the same plan until EOS, a stop condition, or the token limit.

Next: zoom in on one decode epoch and its state update

Decode: Reuse and Advance Provider-local KV/Model State



- ▶ “Provider-local state” means the attention K/V plus any recurrent/convolution tensors produced by that Provider’s assigned layers; P0, P1, and P2 therefore keep different tensors.
- ▶ NDN carries authenticated token feedback, activations, and state identity—not the large state tensors. No discovery, planning, or Selection repeats per token.

Model Identity: Name Is Discovery, Digests Are Authority

- ▶ **model name**: readable discovery, logging, and operator policy;
- ▶ **content digest**: canonical signed manifest binding weights, graph, and configuration;
- ▶ **semantics digest**: tokenizer, preprocessing, generation defaults, output interpretation;
- ▶ **source revision**: provenance only; never substitutes for either digest.

Key point

Reuse requires exact content, semantics, graph, backend, precision, and runtime compatibility—not a matching model name.

Next: make model-specific knowledge replaceable

One Carrier, Multiple Model Families

Adapter port	Responsibility	Forbidden authority
Graph	canonical dependency snapshot	network or device mutation
Splitter	bounded graph-valid candidates	publication or Selection
Task	input/options/result schemas	placement or execution
State	lifecycle and exact state identity	cross-tenant access
Runner	execute accepted ONNX role	changing the plan or cross-Provider transport

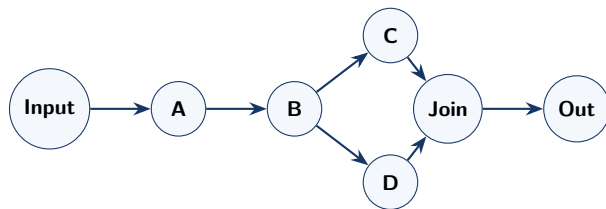
The deployed runtime uses ONNX Runtime; PyTorch/Transformers are confined to a sealed offline exporter.

Key point

An opaque model may expose one atomic node; it remains runnable but is never falsely split.

Next: derive placement structure from a real model graph

Dependency Graph: The Basis of a Legal Split



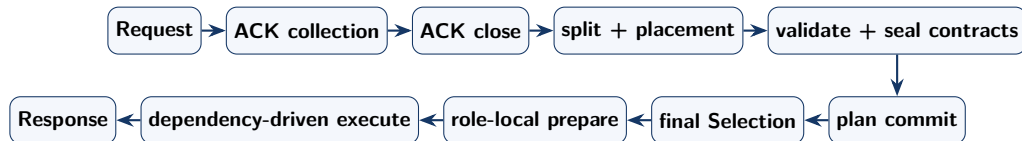
- ▶ nodes and topological order;
- ▶ tensor edges, dtype, shape, size estimate;
- ▶ legal cut edges and collective operations;
- ▶ model input/output contracts;
- ▶ canonical graph digest.

Key point

Each tensor rank is a separate execution role assigned to exactly one Provider; the parallel group is metadata, not a cross-Provider role.

Next: follow one invocation end to end

Complete Lifecycle: One Request, One Final Plan



- ▶ Positive ACK means willingness and bounded capability, not role acceptance or model readiness.
- ▶ After Selection, each Provider fetches canonical ONNX components and assembles one local role.
- ▶ A role issues Interests for its declared inputs and runs after all are verified.

Key point

No preparation-commit round and no global readiness barrier exist.

Next: inspect the request and offer phases

Request Envelope: Signed Intent and Bounded Work

- ▶ binds invocation ID, request ID, attempt, service, model identity, task, input manifest;
- ▶ binds deadline, security domain, encoding version, capability version, and acceptance predicate;
- ▶ payloads are canonical, bounded, schema-identified bytes;
- ▶ UserToken and normal NDN SF authorization protect the wire request.

Key point

The request authorizes discovery for this exact operation; it does not authorize any Provider to execute.

Next: define what a positive ACK can safely claim

Signed Provider Offer: Capability Without Role Commitment

Resource and network view

- ▶ Provider identity and boot epoch;
- ▶ one execution slot/device, backend, usable memory;
- ▶ queue depth and wait estimate;
- ▶ measured RTT and bandwidth;
- ▶ expiry, capture time, units, confidence.

Reusable assets

- ▶ loaded role / assembled role / canonical component residency;
- ▶ lease/pin capability and generation;
- ▶ bounded reusable-state digests;
- ▶ artifact/runtime compatibility evidence.

Key point

ACK evidence is fresh, signed, request-bound, and sanitized before policy code sees it.

Next: freeze the candidate set before planning

ACK Closure: The Planning Linearization Point

- ▶ ACK collection closes exactly once at the discovery deadline.
- ▶ The ordered successful candidate set receives a canonical digest.
- ▶ Late ACKs cannot mutate the snapshot.
- ▶ Repeated reads return the same snapshot.
- ▶ Tokens, keys, leases, and mutable Core handles are removed from the policy view.

Key point

Every placement decision is reproducibly bound to one immutable ACK-closed digest.

Next: invoke one narrow, replaceable planning extension

Placement Extension: Pure Decision, No Side Effects

```
class ModelPlacementStrategy(ABC):  
    name: str  
    version: str  
    state_digest: str  
    @abstractmethod  
    def plan(self, request: PlacementRequest) -> PlacementDecision:  
        ...
```

- ▶ Input: model, graph, bounded candidates, sanitized Providers, network/catalog estimates, objective, and state contracts.
- ▶ Output: one certified split, one Provider per role, fallbacks, digests, and evidence.
- ▶ The implementation is local, allowlisted, digest-pinned operator-trusted code.

Key point

Policy output is revalidated as untrusted data; the Python interface is not a sandbox.

Next: show the default policy

Default Policy: ACK-Driven PreSplitFirst

1. reject model, graph, backend, resource, freshness, and role mismatches;
2. prefer an exact feasible certified split candidate;
3. score loaded roles, assembled roles, canonical layers, network, and queue;
4. otherwise derive a new graph-valid split from the closed offers;
5. rank feasible bindings by predicted end-to-end critical-path cost;
6. break ties with canonical digests for deterministic replay.

Key point

PreSplitFirst remains the default; layer reuse is one cost signal, not a separate LayerReuseFirst architecture.

Next: explain why byte size alone is unsafe

Capacity-Safe Splitting: Runtime Peak, Not File Size

$$M_{\text{peak}} = M_{\text{weights}} + M_{\text{activations}} + M_{\text{state}} + M_{\text{workspace}} + M_{\text{transient}} + M_{\text{margin}}$$

- ▶ each role must fit the Provider's usable envelope;
- ▶ graph cuts must preserve dependency and tensor contracts;
- ▶ homogeneous stages use deterministic near-equal balancing only when safe;
- ▶ missing safe bounds yields NO_FEASIBLE_SPLIT, not optimistic admission.

Key point

“30 GB model / three 12 GB GPUs” is not a valid memory proof.

Next: turn cold-start cost into exact future reuse

Cache Residency: Evidence Has a Strength Order

LOADED ORT ROLE > ASSEMBLED ROLE > CANONICAL LAYERS >
REPOSITORY FETCH > COLD ASSEMBLY

- ▶ Residency is Provider state, not a property of the catalog.
- ▶ Boot epoch, cache generation, expiry, identity, precision, and runtime ABI must match.
- ▶ Loaded-runtime identity also binds Provider boot/process/device and assembler/ORT ABI.

Key point

Warm reuse is an optimization guarded by exact evidence; clean loading remains the correctness fallback.

Next: separate immutable artifacts from volatile residency

Pre-Split Catalog and Distributed Repository

Catalog owns

- ▶ canonical ONNX graph/initializer/tokenizer manifests;
- ▶ certified split candidates without Provider bindings;
- ▶ role/rank interfaces, collective DAG, size and backend contracts;
- ▶ activation, retirement, and revocation snapshots.

Distributed repository owns

- ▶ signed named-object storage and retrieval;
- ▶ segmentation, checksums, replication evidence;
- ▶ deterministic canonical artifact Data names;
- ▶ Provider fetch before local ONNX assembly.

Key point

Persistent model artifacts use the repository; request-scoped tensors use the producing Provider's namespace.

Next: seal the validated decision into the generic collaboration

Plan Commit: Immutable Role and Dataflow Projection

- ▶ binds the exact ACK-closed digest;
- ▶ every role appears once; every Provider receives at most one role;
- ▶ each assignment binds one RoleAssemblySpec;
- ▶ mayPublish, mustFetch, and waitFor form its RoleDataflowContract;
- ▶ the first valid commit digest is immutable;
- ▶ failed validation emits zero Selection messages.

Key point

Plan commit is not Provider acceptance and does not consume a ProviderToken.

Next: make each final Selection crash-atomic

Final Selection: Complete Role Authority

- ▶ exactly one complete execution role per selected Provider;
- ▶ rank, assembly recipe, canonical components, dependencies, budgets, generation;
- ▶ exact publish/fetch/wait name templates and collective rounds;
- ▶ protected by the selected Provider's one-time ProviderToken;
- ▶ Provider verifies request, attempt, plan, role, artifact, dataflow, and security bindings;
- ▶ GPU execution-slot admission occurs JIT, not during ACK.

Key point

Execution authority begins only after the exact Selection is durably accepted.

Next: prepare each role independently

Role-Local Preparation: No Global Barrier

Latch	Meaning
Selection accepted	exact role authority is durable
Local model ready	canonical components, RoleAssemblySpec, ORT ABI, JIT slot, and state verified
Direct inputs ready	every exact Interest target in waitFor verified

A role becomes runnable only when all three latches hold for the same attempt, plan, and generation.

Key point

Each Provider assembles one local ONNX role; consumers pull every cross-Provider input by Interest.

Next: show the data-driven trigger

Data-Driven Execution: Consumer Pulls by Interest

$$\text{Runnable}(r, g) = \text{Accepted}(r, g) \wedge \text{LocalReady}(r, g) \wedge \bigwedge_{d \in \text{DirectInputs}(r)} \text{Verified}(d, g)$$

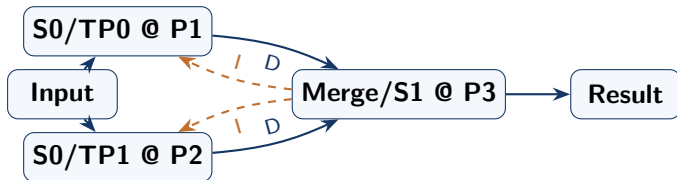
- ▶ `mayPublish`: exact output name templates owned by this role;
- ▶ `mustFetch`: exact Interests this role must express;
- ▶ `waitFor`: exact ALL/ANY/quorum readiness rule (collectives normally ALL);
- ▶ `retransmission` re-expresses the same name; stale attempt/plan/group data fails closed.

Key point

No hidden TCP/RPC push, shared file, or cross-Provider opaque NCCL path may replace this contract.

Next: generalize beyond a linear pipeline

One Scheduler for Pipeline and Tensor Parallel



I = exact-name Interest issued by the consumer role; D = signed tensor Data returned by the producer role.

- ▶ each rank is a distinct role on one Provider;
- ▶ each round names producer, group/epoch, operation, rank, tensor, and segment;
- ▶ peers issue Interests for exact partials and advance only after the declared set arrives;
- ▶ missing rank, stale epoch, ambiguous producer, or unsatisfied join is rejected/fails closed.

Key point

A parallel group is metadata; if merge needs computation, merge is another execution role.

Intermediate Tensors: Deterministic NDN Names

Producer-owned namespace

```
/<producer>/NDNSF/DI/TENSOR/<request>/<attempt>/<plan>/  
<group>/<epoch>/<op>/<round>/<src-role>/<tensor>/<microbatch>/<digest>/<segment>
```

- ▶ consumer first fetches a signed TensorObjectManifest;
- ▶ manifest binds producer/consumer contract, dtype/shape/layout, segment count, and digests;
- ▶ consumer pipelines exact segment Interests and verifies before reassembly;
- ▶ duplicate/out-of-order Data is safe; wrong name, signer, digest, or epoch fails closed.

Key point

Request ID alone is insufficient: attempt, plan digest, group epoch, operation, rank, and segment prevent stale reuse.

Next: define exactly one admissible final result

Result Contract: Events Are Not the Final Answer

- ▶ sealed before execution: terminal role(s), aggregation rule, output schema, acceptance predicate;
- ▶ binds model/task/input/plan/attempt identities and result digest;
- ▶ unary execution publishes no partial result and completes with one Response;
- ▶ streaming execution may publish ordered, attempt-bound Event Data, but no event is the terminal result;
- ▶ EOS, stop, token bound, cancellation, or failure produces a terminal stream state; Response is accepted only after the complete result passes the sealed contract.

Key point

Streaming changes when output becomes visible; it does not weaken the complete-result contract or the single terminal-Response invariant.

Next: separate durable intent from bounded retransmission

One Plan Governs Each Generation Attempt

Setup once

1. Send one Request.
2. Collect Provider ACKs.
3. Choose Providers and layer ranges.
4. Send Selection to assign the work.



Repeat for each decode epoch

1. Read the newest input and saved state.
2. Generate one token.
3. Send the token to the application.
4. Continue, or finish with End and Response.

- ▶ One healthy attempt keeps the same request ID, accepted ACK set, and plan for the whole generation.
- ▶ Every Data name identifies the attempt, role, and decode epoch. Lost Data is requested again by the same exact name; late or wrong-step Data is rejected.
- ▶ A different Provider assignment creates one new attempt inside the same application invocation. At most one terminal result wins.

Key point

Discovery and placement happen once per attempt, not once per token. Recovery does not create a second application request.

Next: compose independent state machines

Orthogonal State Machines, One Auditable History

Machine	Principal states
Invocation	open → terminal success / failure / cancelled / expired
Attempt	discovering → ACK closed → planned → executing → terminal
Plan commit	open → committing → committed or terminal failure
Provider transaction	none → prepared → applied → finalized / rolled back
Role	selected → preparing → ready → running → terminal
Interest/Object	expected → expressed/fetching → verified or rejected
Parallel group	planned → round n → complete or epoch-failed

Key point

Every transition is monotonic, digest-bound, deadline-bound, and replay-safe.

Next: classify failures by the authority that can react

Failure Taxonomy and Required Reaction

Failure class	Required reaction
Invalid offer, split, or policy output	exclude candidate; emit no Selection
Canonical fetch / local assembly failure	fail role/attempt; never emit partial downstream output
Selection delivery unknown	reconcile or retry identical protected bytes
Local load/state miss	use sealed clean-compute fallback or replan
Interest timeout/corruption/rank loss	fail affected role/group/attempt; fence the epoch
Provider crash/partition	fence generation; reconcile durable transaction
Result mismatch	reject Response; terminal failure or new attempt

Next: bound replanning and compensation

Replanning, Compensation, and Old-Output Adoption

- ▶ Replanning creates a new attempt with new assignment and generation identities.
- ▶ Old roles are cancelled or fenced; reservations and pins are released.
- ▶ Already published outputs are not silently reused.
- ▶ Adoption is allowed only if the new plan explicitly accepts the exact object contract and digest.
- ▶ Irreversible external side effects require application-level compensation.

Key point

Recovery never lets a late old output cross an attempt boundary by accident.

Next: make cancellation and deadlines converge

Cancellation, Deadlines, and Resource Release

- ▶ one total invocation deadline continues through discovery, planning, fetch, assembly, loading, and execution;
- ▶ cancellation fences planner results, future commits, roles, objects, and Response;
- ▶ the first durable terminal state wins through compare-and-set;
- ▶ release covers execution-slot admission, cache pins, group keys, local assembly, tensor segments, and handles;
- ▶ unresolved delivery becomes explicit UNKNOWN, never implicit success.

Key point

Safety is preserved even when liveness is lost under a permanent partition.

Next: trace the security chain

End-to-End Security Chain

1. Controller-signed encrypted permissions authorize requester and Provider identities.
2. NAC-ABE routes Request/Selection to service attributes and ACK/Response to permission attributes.
3. UserToken binds ACK and Response to the request.
4. one-time ProviderToken binds final Selection to the chosen Provider.
5. a Provider-specific grant binds request, attempt, plan core, policy snapshot, and protection epoch.
6. group capabilities disclose only the local wrapped epoch key; keys/nonces bind the exact role, peer, tensor manifest, and Data name.
7. ProtectedRuntime rejects mismatches before ORT and zeroizes device then host plaintext on cancel/revoke.

Key point

No planning optimization bypasses NDNF authorization, token, signature, or replay controls.

Next: limit what resource and cache evidence reveals

Privacy of Resource and Cache Metadata

- ▶ expose bounded summaries, not device handles or unrestricted inventories;
- ▶ publish digests, tier, bytes, freshness, and capability—not prompts, token IDs, plaintext state, or tenant names;
- ▶ retain raw tokens and keys only in protected runtime state;
- ▶ encrypt protected Selection retry material at rest and expire it;
- ▶ deny cross-tenant inference-state reuse by default.

Key point

The placement policy receives enough evidence to decide, but not enough authority to inspect or mutate Provider state.

Next: quantify the model state kept by each Provider

Model-State Memory for One Context on Each Provider

Qwen3.6-27B; fixed three-Provider split; batch size 1

Context = prompt tokens + generated tokens still kept in memory.

Provider and layers	10K-token context	100K-token context	256K-token context
P0: Stage0 [0, 21)	245 MiB	1.96 GiB	5.05 GiB
P1: Stage1 [21, 42)	245 MiB	1.96 GiB	5.05 GiB
P2: Stage2 [42, 64)	284 MiB	2.34 GiB	6.05 GiB

- ▶ State grows with context: P0/P1 have 5 full-attention layers; P2 has 6.
- ▶ At 256K, FP16 weights plus state need about 22.3 GiB on P0, 19.9 GiB on P1, and 24.0 GiB on P2.
- ▶ A 32 GiB RTX 5000 Ada fits these static totals, but P2 leaves about 8 GiB for runtime memory. It is suitable for short-context validation; full 256K remains unverified.

256K means the native 262,144-token limit; the YaRN extension is not evaluated. Each concurrent sequence needs its own state.

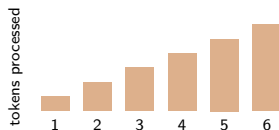
Estimate assumes batch size 1, FP16 KV/convolution state, FP32 recurrent state, and the shown 21/21/22-layer split.

Next: explain why each Provider must keep saved state

Why Saved State Matters

Without saved state

To generate token n , process the prompt and all earlier tokens again.



Repeated work grows after every new token.

- ▶ Each Provider keeps the attention KV, recurrent, and convolution state produced by its own layers.
- ▶ New state is committed only after the Provider successfully publishes its output; a failed step discards it.

With saved state

Process the prompt once. Each later round uses one new token and the saved state.



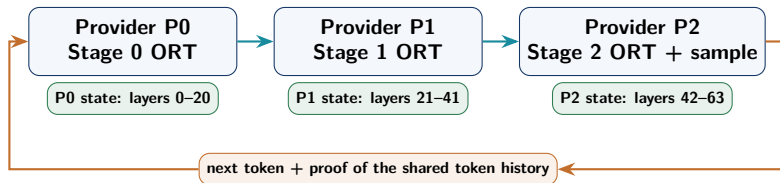
Earlier tokens still affect the result through the saved state.

Key point

Saved state avoids rebuilding earlier model state; it does not remove the LLM's dependence on earlier tokens.

Next: show what moves between Providers and what stays local

What Moves Between Providers—and What Stays Local



- ▶ Each Provider keeps the state produced by its own model layers. These three states are different and cannot replace one another.
- ▶ All roles verify the same token-history digest, token count, and next position. Their local state tensors remain different.
- ▶ NDN carries authenticated activation and token-feedback Data. It does not carry the large saved-state tensors.
- ▶ P2 publishes the external token Event and the internal feedback Data separately.

Key point

A shared proof of token history keeps three different Provider states synchronized without transferring those states over NDN.

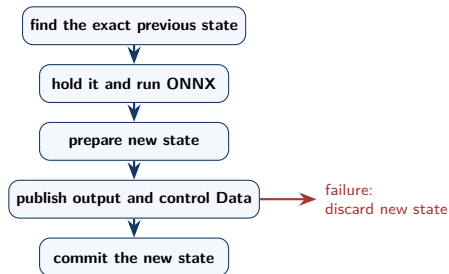
Next: show when saved state is safe to reuse

When Saved State Can Be Reused Safely

Reuse state only when all conditions match

- ▶ model artifact and tensor layout;
- ▶ Provider process and assigned layers;
- ▶ runtime and security context;
- ▶ request attempt and token prefix;
- ▶ exact previous decode epoch.

One safe token-generation step



Key point

A failed step discards its new state. Completion, cancellation, deadline, or Provider restart makes old request state unusable and removes it.

Next: separate perceived latency from decode throughput

Continuous Generation: Optimize TTFT and TPOT

Critical-path metrics

TTFT = discovery + planning + role preparation + prefill + first Event Data

TPOT $\approx$ stage ORT + activation Interest/Data + sampling + feedback Interest/Data

20 token/s target $\Rightarrow$ **TPOT** $\leq$ 50 ms

Optimization order

1. keep ORT sessions, device buffers, and I/O bindings resident;
2. prefill once; decode one token with the exact local saved state;
3. pre-express exact activation, feedback, and Event Interests;
4. sign/publish through a bounded asynchronous queue with backpressure;
5. grant one stream content key, then protect events symmetrically—no per-token ABE bootstrap.

Key point

Streaming improves perceived latency; only incremental stateful decode and a shorter per-epoch path improve tokens/s. This is a target, not current performance evidence.

Observability: Evidence for Decisions and Execution

- ▶ planning input digest, strategy identity, candidate rejection reasons, selected cost components;
- ▶ ACK closure, plan commit, Selection transaction, role readiness, object verification, and terminal status;
- ▶ canonical fetch, local assembly, ORT load, Interest wait/retransmit, collective rounds, compute, TTFT, latency, tokens/s;
- ▶ sampled request-correlated traces with no plaintext prompt or secret material;
- ▶ `report_operation_status()` reports authenticated progress, not execution authority.

Next: show the implemented generic carrier calls

Operator Configuration: Policy, Not Deployment

```
schema: ndnsf-di-application/v2
identity:
  requester: /ndnsf/di/user
  controller: /ndnsf/controller
repository:
  prefix: /ndnsf/di/models
planning:
  strategy: {name: pre-split-first, version: "3",
            digest: "sha256:<strategy-code-and-state>"}
  gpu_safety_margin: 0.15
cache:
  canonical_onnx_components: {retain_verified: true}
  derived_state:
    exact_prefix_local_reuse: true
    cross_tenant_reuse: false
    migration: disabled
```

No model, Provider list, role graph, private key, or precomputed deployment belongs here.

Next: map the design to current code

Implemented Surface and Current Qualification Boundary

Area	Current mechanism or explicit target boundary
Generic carrier	deferred begin, immutable ACK closure, one-shot plan commit
Planning	common graph/candidate values, read-only catalog snapshot, external allowlist, default pre-split-first policy
Adapters	composed graph/split/task/state/runner ports with digest pins
Artifacts	canonical ONNX graph/initializer manifests; repository store/fetch; Provider-local assembly
Execution	one role per Provider; Interest pull; manifest/segment and collective-round gates
State	exact prefix-and-epoch identity, local reuse binding, clean-compute/replan fallback
Delivery	unary terminal Response and request-scoped ordered Event Data plus one terminal Response are implemented
Recovery/security	durable selection transaction, generations, cancellation, tokens, NAC-ABE

Key point

Multi-role Provider, cross-Provider logical role, DEVICE_SET, and LayerReuseFirst-default paths are migration drift, not target completion evidence.

Validation Pyramid: Cheapest Correct Gate First

1. pure contract and negative unit tests;
2. native build and binding checks;
3. one-container security/control smoke with tiny payload;
4. exact-SIF CPU small-ONNX pipeline/tensor/hybrid Interest/Data flows;
5. MiniNDN multi-Provider lifecycle, negative names/epochs, then Qwen complete answers;
6. optional Slurm/Apptainer network, CUDA, stage, and campaign gates.

Key point

A higher-cost test does not repair a failed lower gate, and a lower gate does not prove a higher-layer claim.

Next: report the accepted local evidence

CPU Check: Reuse State Instead of Reprocessing Old Tokens

Fixture: 1-token input context (token 3) → 8 generated tokens.

Both paths generated 4, 5, 6, 7, 8, 9, 10, 2.

Reuse saved state

Prefill reads the 1-token prompt; seven decode rounds each read one new token.

$$1 + (7 \times 1) = 8$$

Recompute the full history

The complete prefix grows from 1 to 8 tokens.

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 = 36$$

Key point

For this 1-token context, saved state avoided 28 repeated token-position computations:
 $36 - 8 = 28$.

This is a correctness check with a tiny CPU ONNX model. It does not measure speed, GPU memory, or Qwen3.6-27B throughput.

Next: separate verified behavior from remaining deployment work

Current Evidence Boundary

Verified with the tiny CPU ONNX model

- ▶ one Request and plan produce ordered token Events, End, and one final Response;
- ▶ focused one-, two-, and four-Provider flows cover cancellation, deadline, tamper/replay rejection, and cleanup;
- ▶ Provider-owned state produces the same output as full-prefix recomputation while avoiding 28 prior-token positions.

Not yet verified

- ▶ GPU-resident KV/model state: the current ORT path still copies saved state into CPU buffers;
- ▶ a real Python application running across separate processes;
- ▶ Qwen3.6-27B GPU correctness, memory use, latency, and throughput.



Key point

CPU tests validate semantics; GPU tests must validate device residency and performance.

Remaining Physical Requalification Boundary

- ▶ exact locally qualified SIF digest;
- ▶ actual allocated GPU model, UUID, memory, CUDA provider, and `cpuFallback=0`;
- ▶ allocation-scoped cross-node NFD connectivity;
- ▶ role assembly hashes, complete graph/initializer coverage, and tensor-edge receipts;
- ▶ exact standalone reference tokens and full distributed answers;
- ▶ warmup-separated TTFT, per-token latency, total latency, and tokens/s distributions.

Key point

These are test-environment facts, not missing semantics in the NDNF-DI lifecycle.

Next: build the local container safely

Local SIF Step 1: Build Inside the Target ABI

- ▶ The release candidate begins as one complete locally built Apptainer SIF.
- ▶ Build native Python/C++ extensions inside that SIF or an ABI-identical sealed builder.
- ▶ Record source seal, base digest, Apptainer, Python SOABI, glibc, Boost, ndn-cxx, NDN-SVS, ORT/CUDA, and SIF SHA-256.
- ▶ Never copy a host-built `_ndnsf.so` into the candidate.
- ▶ Keep models, NDN keys, credentials, and evidence outside the SIF.

Key point

The deployment SIF contains ONNX Runtime, not PyTorch/Transformers; those belong only in the offline exporter.

Next: qualify the exact local candidate

Local SIF Step 2: Qualify the Exact Candidate

- ▶ Import `ndnsf._ndnsf`; run the real MiniNDN runner `-help`.
- ▶ Verify extension/native hashes, `RPATH/ldd` closure, SOABI, library versions, ORT providers, and absolute artifact paths.
- ▶ Run CPU small-ONNX Request/ACK/Selection and Provider-local assembly.
- ▶ Run pipeline and two-rank tensor Interest/Data flows plus negative name/epoch/rank tests.
- ▶ Reject stale extensions, host-library leakage, unresolved symbols, and cwd-dependent artifacts.

Key point

Host tests or a different SIF cannot qualify the file that will run on Tiger.

Next: discover live cluster facts before upload

iTiger Step 1: Read-Only Preflight

```
uofm-vpn-status  
ssh -o BatchMode=yes itiger 'hostname; whoami'  
  
tools/ndnsf-di/ndnsf-di-itiger-qwen discover --help
```

- ▶ Recheck quota, partition, GRES spelling, Apptainer, driver, addresses, and scratch.
- ▶ Keep durable releases/models/evidence under /project/\$USER/ndnsf-di.
- ▶ Never build, convert, download a large model, or infer on the login node.

Next: upload the already-qualified immutable SIF

iTiger Step 2: Hash-Promote the Local SIF

1. Upload the locally qualified SIF to project staging; do not rebuild/materialize another candidate on Tiger.
2. Verify source seal, manifest, and SIF SHA-256; atomically promote the exact file.
3. Preflight sbatch: absolute SIF/bundle/model/evidence paths, cwd, binds, partition/GRES, Provider identities, and output directory.
4. In a GPU allocation, run `apptainer exec -nv`; verify device, driver, ONNX Runtime CUDA, NFD, and NDNSF imports.
5. Assert PyTorch/Transformers are absent and CPU fallback is disabled.

Key point

Host PASS, exact local-SIF PASS, and Tiger GPU PASS are separate gates bound to one SIF hash.

Next: advance through the Qwen test ladder

iTiger Step 3: Network, Reference, Stages, Then DI

1. **multi-node NFD probe**: no model; prove job-scoped faces and signed Data exchange;
2. **standalone reference**: canonical ONNX, tokenizer, prompt set, greedy outputs;
3. **role load**: one Provider per role, exact graph/initializer coverage, ORT CUDA, no CPU fallback;
4. **full development invocation**: request/attempt/plan/group, assembly, Interest/Data edge and collective receipts, complete answer;
5. **formal campaign**: frozen SIF/model/strategy/Provider count; warmup-separated repetitions.

Key point

Run stages in this order; stop at the first failed gate and preserve it.

Next: retain evidence and enforce stop rules

iTiger Step 4: Evidence, Distribution, and Stop Rules

- ▶ bind model/tokenizer, source, exact SIF digest, plan/group, role map, GPU UUIDs, job/run/repetition IDs;
- ▶ retain full answers, tokens, EOS, TTFT, latency, tokens/s, assembly, Interest wait/retransmit, collective and transfer timing;
- ▶ retain failed, timed-out, cancelled, mismatched, and truncated repetitions; exclude them from success summaries without deleting them;
- ▶ never silently retry a measured identity, change GPU count/type, quantize, extend time, or overwrite a failure;
- ▶ promote evidence before allocation scratch is removed.

Key point

One warm run is an anecdote; the frozen repeated campaign is the distribution.

Next: close with the governing design principle

Persistent Artifacts vs. Request-Scoped Tensors

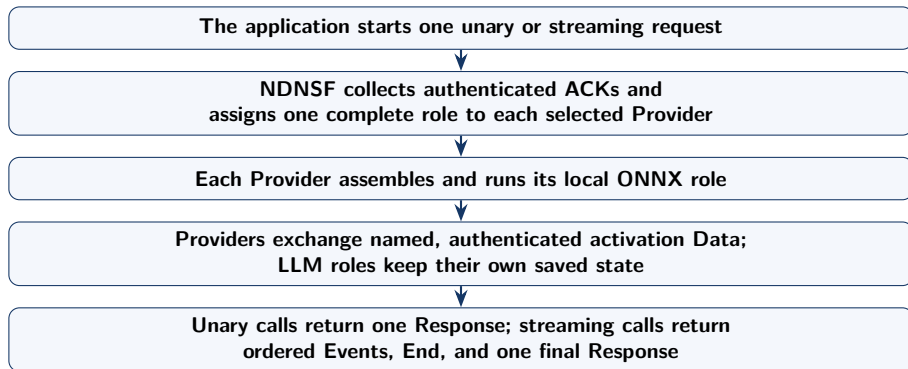
- ▶ DistributedRepo owns persistent canonical ONNX graph/initializer/tokenizer objects: queued store, manifest trust, segmented Data, receipts, activation, recovery, and GC.
- ▶ Request-scoped activations and collective partials are published under the producing Provider's namespace and pulled directly by exact Interests.
- ▶ Both paths use signed segmented NDN Data, but their names, deadlines, authorization, freshness, and retention differ.
- ▶ Repo success does not prove pipeline/tensor dataflow; the corrected protocol needs its own exact-SIF and MiniNDN gates.

Required separation

Model artifacts are durable and content-addressed. Intermediate tensors are attempt/plan/group-epoch bound, short-lived, and accepted only by roles whose sealed `mustFetch/waitFor` contracts name them.

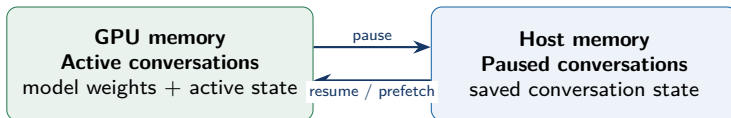
Next: close the corrected protocol before large-model performance claims

How the Complete Request Runs



One request. One plan per attempt.
ONNX state stays local; exact NDN Data moves between roles.

Discussion: More Conversations with Tiered Model State



The same policy runs independently on P0, P1, and P2; each Provider moves only the state produced by its own layers.

- ▶ Model weights normally remain GPU-resident. Large Provider-local state tensors do not move through NDN or between Providers.
- ▶ A scheduler keeps the next conversations on the GPU and asynchronously prefetches a paused conversation before its next decode epoch.
- ▶ This can preserve more conversations than GPU memory alone, but resuming adds host-to-GPU transfer latency; frequent swapping can reduce throughput.

Key point

Tiered state is a feasible future optimization for multi-conversation serving. Its admission, eviction, prefetch, latency, and throughput tradeoffs still require measurement.